

You are working inside the existing `AG_Foundational_Model` repository.

Your task is to perform a complete engineering validation of the entire repository, identify every incomplete or incorrect implementation, fix all defects, add missing tests, and verify that the project contains functioning code for every documented feature.

This is not a documentation review. Treat documentation as a set of claims that must be verified against executable code.

Do not merely report missing functionality. Implement and repair it whenever possible, then test the corrected implementation.

The repository itself is the deliverable.

1. Primary objective

Determine whether the repository has complete, executable, tested code for all required project capabilities:

1. Agricultural corpus manifests and governance.
2. RGB and multiband data loading.
3. ZIP, nested ZIP, NPY, and GeoTIFF support.
4. Group-disjoint and domain-held-out splitting.
5. Source-balanced and dataset-balanced sampling.
6. Exact and near-duplicate detection.
7. Benchmark contamination checking.
8. Official ViT backbone loading.
9. ImageNet, MAE, DINOv2, and DINOv3 initialization.
10. RGB identity and multispectral input adapters.
11. MIM pretraining.
12. DINO-style teacher-student pretraining.
13. iBOT patch-level self-distillation.
14. Morphology-aware masking.
15. Gram anchoring.
16. Joint SSL training.
17. Sequential training stages.
18. High-resolution adaptation.
19. Distributed training.
20. Gradient accumulation.
21. Checkpointing and exact resume.
22. Experiment logging and provenance.
23. Downstream classification.
24. Object detection.
25. Semantic segmentation.
26. Instance segmentation.

27. Counting.
28. Retrieval and clustering.
29. Open-set recognition.
30. Few-shot and label-efficiency evaluation.
31. Domain-shift evaluation.
32. Corruption robustness.
33. Calibration and selective prediction.
34. Statistical testing.
35. Result aggregation.
36. Experiment matrix generation.
37. Reproducibility bundle export.
38. CLI commands and configuration templates.
39. Packaging and optional dependencies.
40. Continuous integration.

2. Testing authority and restrictions

You are authorized to test extensively.

You may:

1. Run the full automated test suite.
2. Run targeted test subsets.
3. Run static analysis.
4. Run type checking if configured.
5. Run package installation checks.
6. Run CLI help and dry-run commands.
7. Run short synthetic CPU training jobs.
8. Run short synthetic forward and backward passes.
9. Run two-process CPU distributed tests with the Gloo backend.
10. Create temporary synthetic RGB, multispectral, GeoTIFF, detection, segmentation, classification, retrieval, and counting datasets.
11. Test checkpoint save, interruption, and resume.
12. Test optional dependency behavior.
13. Measure code coverage.
14. Add tests when coverage or behavior is insufficient.
15. Fix production code and tests.

Do not:

1. Run full publication-scale pretraining.
2. Run long GPU training.
3. Download or inspect private datasets.
4. Inspect the user's real corpus unless explicitly instructed.
5. Modify or delete real data.
6. download pretrained model weights during testing.
7. access the internet.

8. fabricate benchmark results.
9. weaken tests to make code pass.
10. skip failing tests without a documented external reason.

All functional tests must use synthetic fixtures, mocked model sources, temporary directories, and small CPU models unless a local cached resource already exists and is explicitly safe to use.

3. Required validation method

Follow this sequence.

Stage A. Repository inventory

Inspect:

1. Package structure.
2. CLI registration.
3. Configuration schemas.
4. Model implementations.
5. Trainer implementations.
6. Benchmark modules.
7. Test suite.
8. Scripts.
9. Documentation.
10. Package metadata.
11. CI workflows.
12. Example configurations.

Create an internal feature-to-code traceability matrix.

For every claimed feature, record:

1. Documentation location.
2. Production module.
3. Public function or class.
4. CLI entry point.
5. Configuration fields.
6. Existing tests.
7. Validation status.
8. Missing implementation.
9. Required repair.

Do not rely on filenames alone. Open and inspect the relevant implementation.

Stage B. Incompleteness scan

Search production code, tests, scripts, and configurations for:

```
TODO
FIXME
XXX
HACK
NotImplementedError
pass
raise RuntimeError("not implemented")
placeholder
dummy
mock result
fake metric
temporary
```

Review every match manually.

A `pass` inside an intentionally empty exception class is acceptable. An empty production method is not.

Also detect:

1. Functions returning constant values unexpectedly.
2. CLI commands without handlers.
3. Config fields that are parsed but unused.
4. Model parameters that never affect forward computation.
5. Loss values that are logged but not optimized.
6. Checkpoint state that is saved but not restored.
7. Metrics that use the wrong averaging.
8. Optional imports that fail at module import time.
9. Documentation claims without code.
10. Code paths without tests.
11. Test-only implementations accidentally used in production.
12. Silent fallbacks to random initialization.

Fix every genuine defect.

4. Environment validation

Perform these checks.

1. Create or use an isolated environment.
2. Install the package in editable mode with core dependencies.
3. Install development dependencies.

4. Install benchmark and optional dependencies when available.
5. Verify both package entry points.
6. Verify imports from a clean working directory.
7. Verify Python version compatibility declared by the package.
8. Check dependency conflicts.
9. Check missing package declarations.
10. Check platform-specific imports.

Run:

```
python -m ag_foundation --help
ag-foundation --help
```

Every documented command must appear.

Test imports with optional packages absent. Core imports must still work.

5. Static validation

Run the repository's configured static checks.

At minimum:

```
python -m ruff check .
python -m ruff format --check .
```

If the project includes type-checking configuration, run the configured type checker.

Also validate:

1. YAML syntax.
2. JSON schema files.
3. TOML metadata.
4. Shell script syntax.
5. PowerShell script syntax where tooling permits.
6. Broken relative imports.
7. Circular imports.
8. Duplicate CLI registrations.
9. Dead configuration options.
10. Invalid default paths.

Fix all actionable failures.

Do not reformat unrelated files unless needed.

6. Full automated test suite

Run the complete suite.

```
python -m pytest -ra
```

Then run again with coverage.

```
python -m pytest --cov=ag_foundation --cov-report=term-missing --cov-report=xml
```

Requirements:

1. No unexpected failures.
2. No unexpected errors.
3. No unexplained skips.
4. No warnings indicating deprecated or unsafe behavior.
5. New critical modules should have meaningful behavioral coverage.
6. Overall coverage should not be treated as sufficient when critical branches remain untested.

Review each skip.

Classify skips as:

1. Legitimate optional dependency.
2. Unsupported platform.
3. Missing external hardware.
4. Incorrectly skipped.
5. Test design problem.

Fix incorrectly skipped tests.

7. Configuration validation

Enumerate every example and publication configuration.

For each configuration:

1. Parse it.
2. Resolve relative paths without reading data.
3. Validate unknown-key rejection.
4. Validate type constraints.
5. Validate incompatible options.
6. Validate patch-size divisibility.

7. Validate crop settings.
8. Validate DINO temperature settings.
9. Validate EMA settings.
10. Validate masking probabilities.
11. Validate loss weights.
12. Validate distributed settings.
13. Validate gradient accumulation.
14. Validate sampling strategy.
15. Validate benchmark task compatibility.
16. Validate adaptation mode.
17. Validate stage transition compatibility.

Every configuration should support a command equivalent to:

```
python -m ag_foundation validate-config --config <path>
```

Add this command if missing.

Validation must not construct a large model, download weights, or inspect image data.

8. CLI validation

Test every CLI command.

Required classes of tests:

1. `--help`.
2. Missing required argument.
3. Invalid argument.
4. Valid dry run.
5. Output directory creation.
6. Structured error handling.
7. Exit code correctness.
8. Command logging.
9. Configuration override behavior.
10. No heavy import during help display.

Test commands including:

```
train-mim
train-dino
train-joint
train-pipeline
validate-config
create-catalog
```

```
build-manifest
validate-manifest
build-splits
validate-splits
find-duplicates
check-contamination
slice-geotiffs
create-demo-data
benchmark
evaluate-predictions
aggregate-results
generate-experiment-matrix
export-repro-bundle
```

If a documented command is missing, implement it.

9. Data layer testing

Use temporary synthetic files only.

Test:

RGB

1. JPEG.
2. PNG.
3. Grayscale conversion behavior.
4. Corrupt image handling.
5. Too-small image handling.
6. Deterministic validation crop.
7. Random training crop.

NumPy

1. CHW.
2. HWC.
3. Single channel.
4. Ambiguous channel layout.
5. Invalid channel count.
6. NaN.
7. Infinity.
8. Out-of-range floating values.
9. Integer normalization.

GeoTIFF

1. Single band.
2. Three band.
3. Five band.
4. CRS preservation.
5. Affine transform preservation.
6. Tiling.
7. Edge tiles.
8. Rotated affine transform.
9. Stitching.
10. Unsupported datatype.

Archives

1. ZIP.
2. Nested ZIP.
3. macOS metadata files.
4. Duplicate member names.
5. Invalid archive.
6. Reused archive handles.
7. Relative archive member paths.

Catalogs and manifests

1. Relative paths.
2. Archive member paths.
3. Duplicate sample IDs.
4. Missing fields.
5. Invalid splits.
6. Channel and wavelength mismatch.
7. Stable hashing.
8. Canonical serialization.
9. Version migration.

10. Split and sampling tests

Test all split strategies:

1. Random group-disjoint.
2. Source-held-out.
3. Farm-held-out.
4. Field-held-out.
5. Country-held-out.
6. Season-held-out.
7. Sensor-held-out.

8. Crop-held-out.
9. Observation-held-out.
10. Fixed user split.

Validate:

1. No group overlap.
2. Determinism.
3. Approximate requested proportions.
4. Helpful behavior when exact proportions are impossible.
5. Split hash stability.
6. Empty split rejection.
7. Tiny-group handling.

Test samplers:

1. Proportional.
2. Square-root.
3. Uniform source.
4. User weights.
5. Distributed weighted sampling.
6. Tiny sources.
7. Replacement.
8. No replacement.
9. Epoch reseeding.
10. Effective probability reporting.

11. Governance code testing

Test exact hashing, perceptual hashing, duplicate clustering, and contamination checking with generated images.

Include:

1. Exact duplicate.
2. Recompressed duplicate.
3. Resized duplicate.
4. Cropped duplicate.
5. Visually different image.
6. Duplicate across archives.
7. Duplicate across pretraining and benchmark manifests.
8. Human-review threshold case.
9. Resume from partial metadata.
10. Dry-run mode.
11. No automatic deletion.

Confirm exclusion reports are deterministic.

12. Backbone and adapter tests

Use mocked `timm` models.

Test:

1. ViT S.
2. ViT B.
3. ViT L.
4. Patch size 14.
5. Patch size 16.
6. ImageNet source.
7. MAE source.
8. DINOv2 source.
9. DINOv3 source.
10. Random initialization.
11. Classifier removal.
12. Token output.
13. Positional interpolation.
14. Intermediate feature extraction.
15. Frozen encoder.
16. Full fine-tuning.
17. Adapter-only training.
18. LoRA training.

Adapter tests:

1. RGB identity initialization.
2. RGB output equality at initialization.
3. Four-band input.
4. Five-band input.
5. Additional bands initialized correctly.
6. Gradient propagation.
7. Frozen adapter.
8. Trainable adapter.
9. Checkpoint round trip.
10. Old checkpoint migration.

13. MIM tests

Test:

1. Mask generation.
2. Exact mask ratio where possible.
3. Masked-only loss.
4. No contribution from visible patches.

5. Pixel reconstruction.
6. Normalized-pixel reconstruction.
7. Teacher feature reconstruction.
8. Weighted morphology reconstruction.
9. Batch size one.
10. Several patch grids.
11. Mixed precision.
12. Zero valid-mask protection.
13. Gradient flow.
14. Checkpoint save and load.
15. Resume.

14. DINO tests

Test:

1. Student and teacher initialization.
2. Teacher has no gradients.
3. EMA update.
4. Adapter EMA.
5. Projection head EMA.
6. Global views.
7. Local views.
8. Cross-view pairing.
9. Teacher centering.
10. Temperature scaling.
11. Entropy reporting.
12. Finite outputs.
13. Collapse diagnostics.
14. Checkpoint compatibility.
15. Resume after interruption.

15. iBOT tests

Test:

1. Patch head construction.
2. Masked student predictions.
3. Teacher patch targets.
4. Patch centering.
5. Patch temperatures.
6. Cross-view behavior.
7. Mask-aware reduction.
8. No-valid-patch handling.
9. Head EMA update.
10. Gradient flow.

11. Checkpoint round trip.
12. Legacy DINO checkpoint loading.

16. Morphology masking tests

Test each mask generator.

Random block

- Exact patch count.
- Contiguous blocks.
- Determinism.

Foreground

- Supplied attention map.
- Supplied saliency map.
- Image-based fallback.
- Uniform image.
- All-black image.
- All-white image.

Boundary

- Synthetic leaf-like circle.
- Thin line.
- Edge-free image.
- Multiple disconnected regions.

Mixed

- Probability selection.
- Fixed seed reproducibility.
- Probability validation.

Curriculum

- Beginning.
- Midpoint.
- End.
- Resumed step.

Validate CPU and CUDA device behavior when CUDA is available, without requiring CUDA.

17. Gram anchoring tests

Test:

1. Frozen reference model.
2. EMA reference model.
3. Layer selection.
4. Missing layer error.
5. CLS exclusion.
6. Register-token exclusion.
7. Gram shape.
8. Spatial subsampling.
9. Mixed precision.
10. No reference gradients.
11. Student gradients.
12. Checkpoint round trip.
13. ViT S, B, and L test doubles.

18. Joint SSL tests

Test every valid objective combination.

1. DINO only.
2. iBOT only where supported.
3. Morphology MIM only.
4. Gram only with valid student and reference features.
5. DINO plus iBOT.
6. DINO plus morphology.
7. Full joint objective.
8. Scheduled loss weights.
9. Disabled module nonconstruction.
10. Shared backbone computation.
11. Finite total loss.
12. Correct weighted sum.
13. Metric output schema.
14. Teacher update timing.
15. Checkpoint round trip.
16. Resume.

19. Trainer tests

Use tiny fake models and datasets.

Test:

1. One training step.
2. Two training steps.
3. Validation.
4. Gradient clipping.
5. Mixed precision paths.
6. NaN loss failure.
7. Infinite gradient failure.
8. Empty loader failure.
9. Scheduler stepping.
10. Best checkpoint selection.
11. Last checkpoint replacement.
12. Artifact atomicity.
13. Visualization RNG restoration.
14. Resume from epoch boundary.
15. Resume with optimizer state.
16. Resume with scheduler state.
17. Resume with gradient scaler.
18. Resume with DataLoader generator state.

20. Gradient accumulation tests

Test:

1. Accumulation one.
2. Accumulation two.
3. Final incomplete accumulation group.
4. Loss scaling.
5. Optimizer-step count.
6. Scheduler-step count.
7. Teacher update count.
8. Gradient clipping timing.
9. DDP no-sync use.
10. Effective batch reporting.
11. Checkpoint counters.
12. Resume.

Compare accumulated gradients with an equivalent larger batch where mathematically valid.

21. Distributed testing

Run a short two-process CPU Gloo integration test.

Test:

1. Process initialization.
2. Rank detection.
3. Distributed sampler.
4. Metric reduction.
5. Gradient synchronization.
6. Rank-zero artifact writing.
7. No duplicate checkpoint writes.
8. Barrier use.
9. DDP checkpoint load.
10. Single-process checkpoint load under DDP.
11. DDP checkpoint load in single-process mode.
12. Graceful cleanup.

Use a tiny fake model and no more than two optimizer steps.

If the operating system cannot support the test, retain mocked unit coverage and document the limitation clearly.

22. Sequential and pipeline tests

Test:

1. MIM to DINO.
2. MIM to joint SSL.
3. Joint SSL to high resolution.
4. Gram refinement.
5. Stage order.
6. Incompatible stage rejection.
7. Backbone transfer.
8. Adapter transfer.
9. Head reset.
10. Stage manifest.
11. Stage checkpoint hash.
12. Interrupted pipeline resume.
13. Completed-stage skip.
14. Dry run.

23. High-resolution tests

Test:

1. 224 to 320.
2. 224 to 448.
3. Patch-size divisibility.

4. Positional interpolation.
5. Token-grid reporting.
6. Checkpoint compatibility.
7. No layer reinitialization.
8. Reduced learning rate configuration.

24. Downstream encoder tests

Test foundation checkpoint loading for:

1. Current MIM checkpoint.
2. Current DINO checkpoint.
3. Joint SSL checkpoint.
4. ImageNet model source.
5. MAE source.
6. DINOv2 source.
7. DINOv3 source.
8. Random source.

Test output modes:

1. CLS.
2. Mean patch.
3. CLS plus mean.
4. Intermediate layers.
5. Frozen.
6. LoRA.
7. Adapter only.
8. Full fine-tuning.

25. LoRA tests

Test:

1. Attention projection insertion.
2. Optional MLP insertion.
3. Rank 4.
4. Rank 8.
5. Rank 16.
6. Base weights frozen.
7. LoRA weights trainable.
8. Parameter counts.
9. Merge.
10. Unmerge.
11. Checkpoint round trip.
12. Unsupported target error.

26. Classification tests

Use synthetic separable data.

Test:

1. Linear probe.
2. Attentive probe.
3. Full fine-tuning.
4. Hierarchical labels.
5. Multilabel labels.
6. Top-1.
7. Top-5.
8. Macro F1.
9. Balanced accuracy.
10. Per-class recall.
11. Calibration.
12. Prediction artifact format.

27. Segmentation tests

Use small masks with known expected metrics.

Test:

1. Frozen linear probe.
2. Intermediate feature aggregation.
3. UPerNet decoder.
4. Ignore index.
5. Binary classes.
6. Multiclass masks.
7. mIoU.
8. Dice.
9. Boundary F1.
10. Upsampling.
11. Frozen encoder gradients.
12. Full fine-tuning gradients.

28. Detection and instance segmentation tests

Use synthetic COCO annotations.

Test:

1. COCO parsing.
2. Box conversion.
3. Empty annotations.
4. Invalid boxes.
5. Polygon masks.
6. RLE masks where dependencies exist.
7. ViT feature pyramid output.
8. Faster R-CNN construction.
9. Mask R-CNN construction.
10. One tiny training forward.
11. One inference forward.
12. COCO metric dependency error.
13. Prediction serialization.

Do not perform full detector training.

29. Counting tests

Test:

1. CLS regression.
2. Density prediction.
3. Detection-derived count.
4. MAE.
5. RMSE.
6. MAPE with zero target.
7. R-squared.
8. Bias.
9. Grouped count metrics.

30. Retrieval tests

Use known embeddings.

Test:

1. Cosine similarity.
2. L2 normalization.
3. Self-match exclusion.
4. Query-gallery mode.
5. Class retrieval.
6. Cross-source retrieval.
7. Chunked computation.
8. Recall at K.

9. mAP at K.
10. NMI.
11. Adjusted Rand index.
12. Stable tie ordering.

31. Open-set tests

Use synthetic known and unknown distributions.

Test:

1. Maximum softmax.
2. Energy score.
3. Prototype distance.
4. k-nearest-neighbor distance.
5. Mahalanobis distance.
6. Validation threshold selection.
7. AUROC.
8. AUPR.
9. FPR95.
10. Accuracy at coverage.
11. No test-set threshold tuning.

32. Label-efficiency tests

Test:

1. One percent.
2. Five percent.
3. Ten percent.
4. Twenty-five percent.
5. Fifty percent.
6. Full data.
7. Examples per class.
8. Five subset seeds.
9. Shared subsets across models.
10. Class coverage.
11. Tiny class behavior.
12. Split hash.
13. Area under the label-efficiency curve.
14. Reference score matching.

33. Domain-shift tests

Use synthetic metadata groups.

Validate:

1. Source holdout.
2. Crop holdout.
3. Farm holdout.
4. Field holdout.
5. Country holdout.
6. Season holdout.
7. Sensor holdout.
8. In-domain score.
9. Out-of-domain score.
10. Worst-domain score.
11. OOD drop.
12. Retention.
13. Empty-domain handling.

34. Corruption suite tests

For every corruption:

1. All five severity levels.
2. Determinism.
3. Shape preservation.
4. Valid value range.
5. Input type preservation where promised.
6. Sample ID preservation.
7. No permanent dataset modification.

Validate mean and relative corruption error calculations.

35. Calibration tests

Test:

1. Temperature fitting.
2. Temperature serialization.
3. ECE.
4. Adaptive ECE.
5. Brier score.
6. NLL.
7. Reliability bins.
8. Risk-coverage curve.
9. Fixed coverage metrics.
10. Segmentation selective mIoU.
11. Validation-only fitting.

36. Statistical tests

Test:

1. IID bootstrap.
2. Cluster bootstrap.
3. Paired bootstrap.
4. Wilcoxon signed-rank.
5. Holm correction.
6. Deterministic resampling.
7. Effect size.
8. Missing model-dataset entries.
9. Single-cluster error.
10. Structured JSON and CSV outputs.

Validate statistical routines against small cases with analytically known or externally verifiable results.

37. Result aggregation tests

Test:

1. Average rank.
2. Tied ranks.
3. Task-family win count.
4. Worst-domain score.
5. Label-efficiency area.
6. Global-task mean.
7. Dense-task mean.
8. Compute-normalized score.
9. Missing result handling.
10. Raw metric preservation.
11. Normalization metadata.

38. Experiment matrix tests

Test:

1. Deterministic IDs.
2. Seed expansion.
3. Initialization expansion.
4. Objective expansion.
5. Model-size expansion.
6. Label-fraction expansion.
7. Adaptation expansion.
8. Collision detection.

9. Config inheritance.
10. Dependency graph.
11. Dry-run command output.
12. CSV and JSON output.

39. Artifact and reproducibility testing

Test:

1. Atomic metric writes.
2. Atomic checkpoint writes.
3. Immutable attempt manifests.
4. Command log append behavior.
5. Git metadata.
6. Environment metadata.
7. Corpus manifest hash.
8. Split hash.
9. Checkpoint hash.
10. Prediction hash.
11. Compute metadata.
12. Reproducibility bundle.
13. Missing optional artifact behavior.

40. Short synthetic end-to-end workflows

Run these workflows using generated data and tiny fake backbones.

Workflow 1, MIM

```
Synthetic RGB dataset
→ manifest
→ split
→ one short MIM training run
→ validation
→ checkpoint
→ resume
→ prediction or feature export
```

Workflow 2, DINO

```
Synthetic RGB dataset
→ multi-crop loading
→ two-step DINO run
```

- EMA update
- checkpoint
- resume

Workflow 3, joint SSL

- Synthetic plant-shaped images
- morphology masks
- DINO plus iBOT plus morphology plus Gram loss
- two optimizer steps
- checkpoint
- resume

Workflow 4, distributed

- Synthetic dataset
- two CPU processes
- DDP training
- global metric reduction
- rank-zero checkpoint

Workflow 5, classification benchmark

- Synthetic checkpoint
- frozen encoder
- linear probe
- metrics
- calibration
- prediction artifact

Workflow 6, segmentation benchmark

- Synthetic images and masks
- frozen dense probe
- mIoU
- domain grouping

Workflow 7, detection benchmark

```
Synthetic COCO data
→ feature pyramid
→ one training forward
→ inference
→ prediction serialization
```

Workflow 8, complete experiment reporting

```
Synthetic seed results
→ aggregation
→ bootstrap confidence intervals
→ average ranks
→ reproducibility bundle
```

These workflows must remain short and CPU-safe.

41. Backward compatibility

Verify that existing behavior still works.

1. Existing MIM configurations parse.
2. Existing DINO configurations parse.
3. Existing MIM checkpoints load.
4. Existing DINO checkpoints load.
5. Existing RGB identity adapter remains unchanged.
6. Existing multiband adapter remains supported.
7. Existing demo data command works.
8. Existing command logs remain compatible.
9. Existing run manifests remain readable.
10. Existing resume behavior remains correct.
11. Existing tests still pass.
12. Existing import paths remain valid or have compatibility exports.

42. Documentation-to-code verification

Review every claim in:

```
README.md
docs/architecture.md
```

```
docs/configuration.md
docs/data-formats.md
docs/training.md
docs/experiment-tracking.md
docs/reproducibility.md
docs/testing.md
docs/research-roadmap.md
docs/implementation-map.md
docs/troubleshooting.md
```

For each claim, classify it as:

1. Implemented and tested.
2. Implemented but insufficiently tested.
3. Partially implemented.
4. Not implemented.
5. Research plan only.
6. Incorrect or outdated.

Fix code or documentation so they agree.

Do not convert missing code into vague roadmap language when the feature is required for project completion.

Do not claim that scientific performance has been validated using synthetic tests.

43. CI validation

Inspect and update continuous integration.

CI should test:

1. Core installation.
2. Core unit tests.
3. Benchmark unit tests.
4. CLI help.
5. Configuration validation.
6. Synthetic end-to-end workflows.
7. Optional dependency import behavior.
8. Ruff.
9. Supported Python versions.
10. Linux.
11. macOS.
12. Windows where feasible.
13. No network access.
14. No pretrained downloads.

15. No real data.
16. No long training.

Validate workflow syntax.

Do not trigger remote CI unless explicitly permitted.

44. Performance and resource checks

For synthetic workflows, record:

1. Runtime.
2. Peak memory when available.
3. Number of parameters.
4. Trainable parameters.
5. Samples processed.
6. Optimizer steps.
7. Effective batch size.

Detect obvious performance bugs:

1. Reopening ZIP archives for every sample.
2. Recomputing static embeddings.
3. Retaining computation graphs.
4. Unbounded prediction accumulation.
5. Excessive device transfers.
6. Duplicate model forward passes.
7. Non-streaming hash computation.
8. Rank-wide duplicate artifact generation.

Fix clear defects without premature optimization.

45. Security and safety checks

Inspect for:

1. Unsafe archive extraction.
2. Path traversal.
3. Unsafe YAML loading.
4. Arbitrary code execution through configuration.
5. Unsafe checkpoint loading assumptions.
6. Temporary-file races.
7. Command injection in launch scripts.
8. Accidental secret logging.
9. User-path leakage in released artifacts.
10. Unvalidated output paths.

Add tests for archive path traversal and unsafe manifest paths.

46. Failure-handling tests

Test clear errors for:

1. Missing dataset root.
2. Missing manifest.
3. Invalid checkpoint.
4. Unsupported checkpoint schema.
5. Channel mismatch.
6. Invalid crop size.
7. Empty dataset.
8. One-group split.
9. Missing optional dependency.
10. Invalid annotation.
11. NaN loss.
12. Distributed misconfiguration.
13. Interrupted stage.
14. Corrupt artifact.
15. Invalid prediction schema.

Errors must be actionable and must not silently continue.

47. Repair rules

When a defect is found:

1. Write or update a test that reproduces it.
2. Confirm the test fails for the correct reason.
3. Fix production code.
4. Run the focused test.
5. Run related tests.
6. Run the full suite after all repairs.
7. Record the defect and repair in the final report.

Never change an expected value merely to match incorrect production behavior.

48. Final test sequence

After all repairs, run this validation sequence.

```
python -m ruff check .  
python -m ruff format --check .
```

```
python -m pytest -ra
python -m pytest --cov=ag_foundation --cov-report=term-missing --cov-report=xml
```

Then run the short synthetic end-to-end workflows.

Then run package build checks if build tooling exists.

```
python -m build
```

Install the built wheel in a clean temporary environment and verify:

```
python -m ag_foundation --help
ag-foundation --help
```

Do not perform full model training or use real agricultural data.

49. Required validation artifacts

Create:

```
reports/validation/
  feature_traceability.csv
  feature_traceability.json
  test_inventory.md
  documentation_claims.csv
  failing_tests_initial.txt
  repaired_defects.md
  final_test_results.txt
  coverage_summary.md
  skipped_tests.md
  optional_dependencies.md
  synthetic_workflows.md
  backward_compatibility.md
  unresolved_limitations.md
  validation_summary.md
```

The feature traceability table should include:

```
feature
documentation_claim
production_module
```

```
public_interface
configuration
tests
unit_status
integration_status
synthetic_e2e_status
real_data_validation_required
notes
```

Do not include fabricated scientific metrics.

50. Completion criteria

The project can be marked engineering-complete only when:

1. Every required capability maps to production code.
2. Every public code path has behavioral tests.
3. All full-suite tests pass.
4. Static checks pass.
5. Synthetic end-to-end workflows pass.
6. Checkpoint and resume behavior pass.
7. Distributed CPU integration passes or has a documented platform limitation.
8. Existing functionality remains compatible.
9. Documentation accurately reflects implementation.
10. No required feature remains a placeholder.
11. No real data inspection was required.
12. No full pretraining was required.
13. Scientific claims remain clearly unvalidated until real experiments run.

51. Final response

Return a concise but complete report containing:

1. Overall validation status.
2. Features verified as complete.
3. Features that were incomplete and were repaired.
4. Important files added or changed.
5. Full test-suite result.
6. Coverage result.
7. Static-analysis result.
8. Synthetic end-to-end workflow results.
9. Distributed test result.
10. Package-build result.
11. Skipped tests and reasons.
12. Optional dependencies not available.

13. Genuine unresolved limitations.
14. Exact commands executed.
15. Confirmation that no real dataset was inspected and no full pretraining was run.

Do not claim that the model is scientifically validated or CVPR-ready based only on engineering tests.

The final distinction must be explicit:

Engineering completeness
versus
Scientific performance validation

Engineering completeness should be established through the work in this prompt.

Scientific performance validation will require the real corpus, downstream datasets, publication experiments, and statistical results in a later execution stage.